

Sarvesh Patil

Mechanical Engineer

ABOUT ME

I am a mechanical engineer who learns by designing and building hardware. I take ideas from quick CAD concepts and breadboard tests to validated prototypes, with clear documentation at every step. I enjoy combining mechanics, electronics, and firmware to create reliable assemblies, paying attention to tolerances, manufacturability, and testability so the final design works the way the project scope defines it.



sarveshpatil73.sp@gmail.com



+1 (217) 904-0560



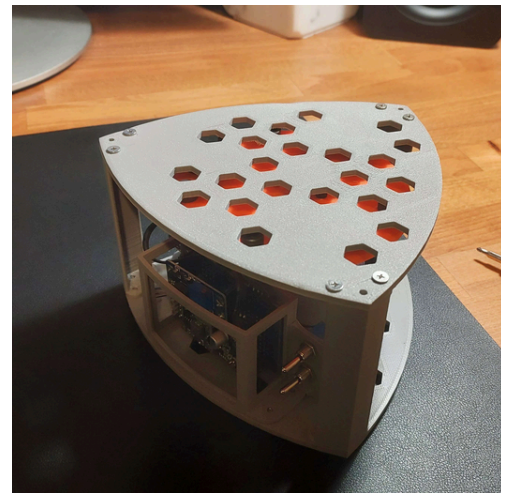
<https://github.com/SarveshPatil7>



<http://www.linkedin.com/in/sarveshpatil07>

Self Balancing Prism

A mobile reaction wheel based inverted pendulum setup that balances on an edge



Completed prototype

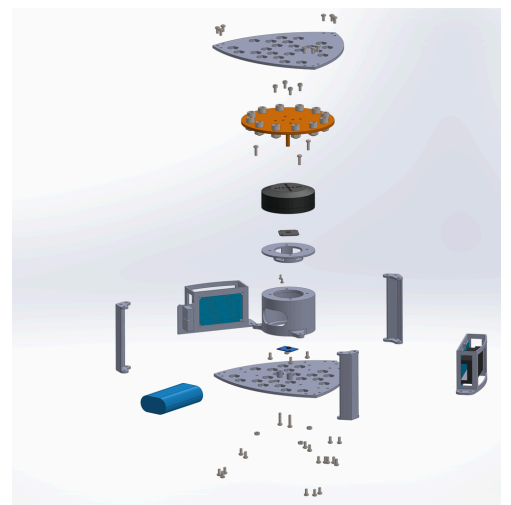
<https://www.youtube.com/watch?v=W3pju5vKch4>

OVERVIEW

I saw some inverted pendulum projects and was further inspired by 'shapes of constant width' and thought it would be fun to build my own version that balances on an edge. This was a personal project with no set goal other than learning. I wanted to design the hardware, write the firmware, and get the whole system working by myself

WHAT I DID

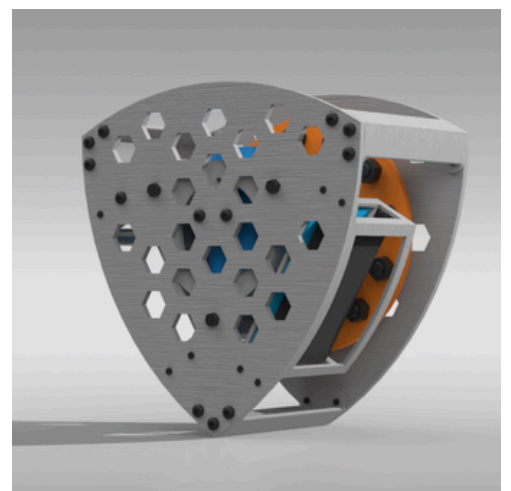
- Designed the assembly in SolidWorks and 3D-printed it with PLA
- Selected and integrated all components (ESP32-S3, GBM5208 BLDC motor, DRV8313 motor driver, MT6701 magnetic encoder, MPU6050 IMU)
- Wrote the full firmware : angle estimation, task scheduling, PID + state observer, and FOC motor control
- Performed experimental tuning, validation, drift suppression, and stability testing



Exploded assembly

HOW IT WORKS

- Dual-core ESP32-S3 running FreeRTOS
- One core handles the IMU sampling
- The other runs the angle estimation, PID and FOC loop
- Angle estimation merges gyro + accelerometer data using complimentary filters and drift correction for stable orientation tracking
- The GBM5208 reaction wheel provides the counter-torque needed to keep the prism upright
- The PID controller is complimented by a state observer for the motor to avoid saturation of the motor speed



Prototype rendering in Solidworks Visualize

Anti-cogging algorithm

OVERVIEW

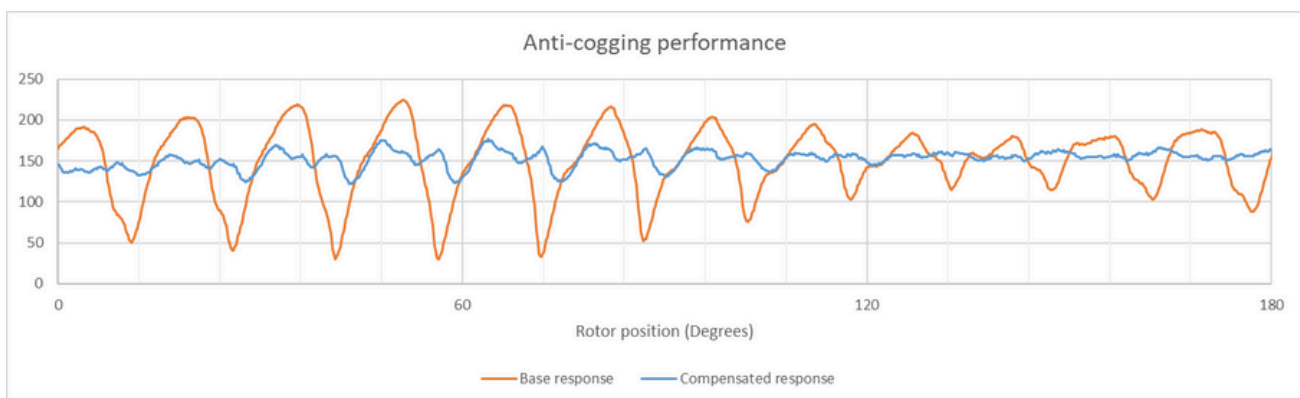
Precise robots need smooth torque, but most BLDC setups still suffer from cogging and torque ripple. High end systems overcome this with skewed stators, high pole count motors, and high ratio gear trains, which works but is too expensive or bulky for compact direct drive and budget sensitive builds. This project is an ongoing attempt to build an open source anticogging library that is lightweight and easy to use for hobby builders and research prototypes, giving them access to cleaner motion without current sensing drivers or external torque sensors.

HOW IT WORKS

- Test setup uses a GBM5208 24n22p gimbal motor driven by a DRV8313 based FOC board
- A 12 bit magnetic rotary encoder feeds position back to an ESP32 running the control code
- The motor is run at very low speed in torque control mode with a constant voltage command
- Velocity is recorded in both directions and averaged to cancel out friction effects
- The averaged data is differentiated to get an angular acceleration profile over one electrical revolution
- That acceleration profile is scaled and used as an offset look up table on the drive voltage to counteract torque ripple

RESULTS

- Implemented an anti-cogging algorithm that works on entry-level hardware, without external torque sensors or current-sensing motor drivers
- Current prototype cuts peak-to-peak torque ripple by roughly 71% in test conditions, with further tuning and validation in progress



FUTURE ROADMAP

- Extend from constant speed calibration to variable speed profiles

Thermal FEA Solver

OVERVIEW

This was one of the projects I made for the class ME 471 - Finite Element Analysis. I was given a basic Python framework to parse Abaqus .inp files and plot results, but the entire solver had to be written from scratch. I used the assignment as a chance to understand the full workflow behind 2D thermal FEA and to implement the complete finite element formulation myself.

WHAT I DID

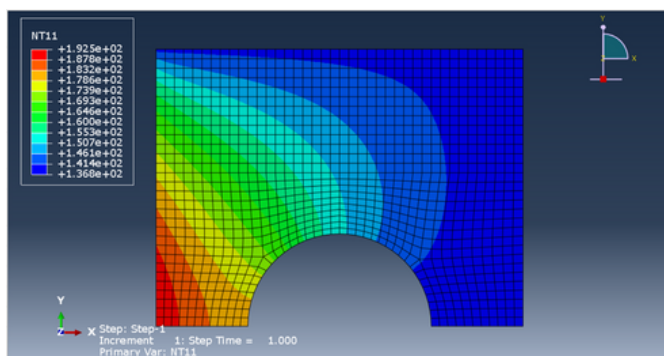
- Wrote the entire solver logic including stiffness matrix formation, element routines, assembly, and boundary condition application
- Implemented conduction, convection, and flux contributions at the element level using the Q4 formulation
- Extended the steady state solver to transient solver using adjustable time stepping schemes
- Developed the time integration workflow including lumped capacitance matrix formation, critical time step calculation, and both Forward Euler and Crank Nicolson updates

HOW IT WORKS

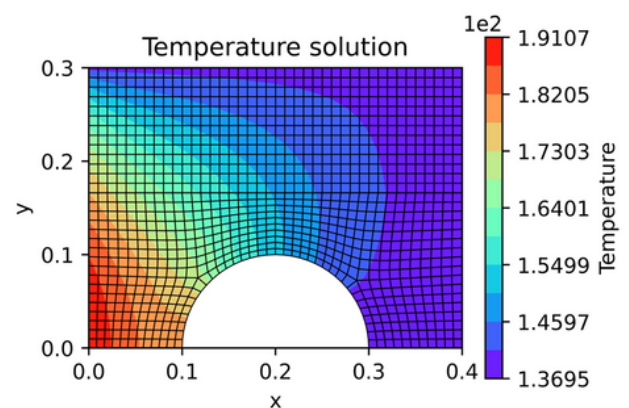
- The provided input reader builds the mesh and boundary data
- My routines compute the element matrices for each Q4 element and assemble them into the global system
- Boundary conditions are applied through system partitioning into free and prescribed nodes
- For transient analysis, the system uses the heat capacity matrix and integrates the temperature field forward in time
- Results are returned to the instructor's visualization script to generate contour plots

RESULTS

- My solver matched Abaqus temperature fields within well under one percent on all test meshes
- Both steady state and transient solutions followed the expected trends across different mesh sizes
- Time history plots from my code and Abaqus were almost identical, which confirmed that the element formulations and boundary conditions were implemented correctly



A. Results from Abaqus



B. Results from the python solver

Comparison of results from Abaqus vs custom coded solver results

Immersion Cooling Electric Vehicle Battery

OVERVIEW

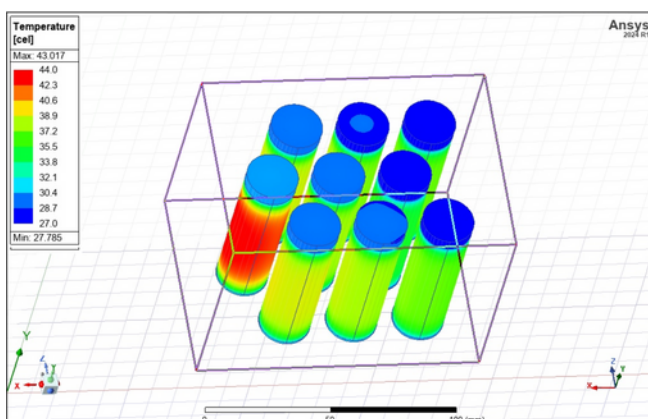
This project was part of my electronics cooling class. Our team studied the thermal limits of the coldplate cooled battery pack used in the 'Lucid Air'. The 21700 cells in the pack struggle during high discharge because the coldplates cannot keep the cell temperatures uniform, which leads to hotspots and a higher risk of thermal runaway. We explored immersion cooling as an alternative and evaluated how much it could improve temperature levels and uniformity.

WHAT I DID

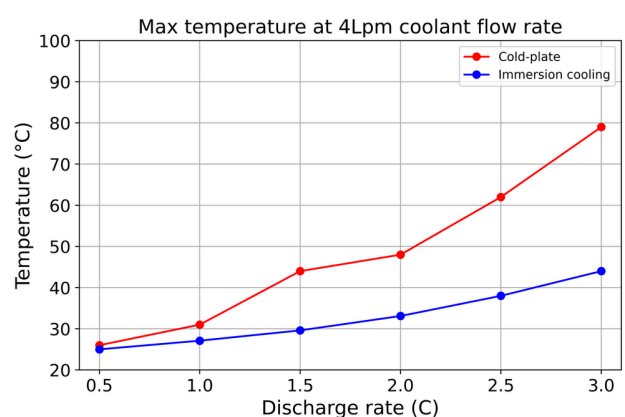
- Recreated the Lucid cell arrangement by reverse engineering the spacing from reference photos using image analysis
- Built the full CAD for the module and the 21700 cells including the internal jellyroll structure
- Modeled both the coldplate design and the immersion cooled version
- Verified coolant properties and gathered the full material set for mineral oil
- Ran a portion of the simulations for 1C to 3C discharge conditions
- Wrote the SOP for the partial immersion setup and helped with the final report and plots

HOW IT WORKS

- Each cell generates heat based on its discharge rate and internal resistance
- In the coldplate design, heat moves into an aluminum plate that carries coolant in a channel, effective mostly in regions in directly in contact with the coldplate
- In the immersion design, mineral oil removes the heat from the entire surface of the cell for a more uniform effect
- The simulations track maximum temperature, temperature spread, and how flow rate affects both
- The geometry and heat models were built by our team and simulated in Icepak



A. Ansys Icepak simulation



B. Comparing the performances

Model-based Parameter Tuning for High Dimensional Systems

Dimension Reduction using Deep Learning for High-Dimensional Reliability Analysis

OVERVIEW

In my Data Driven Design Methods coursework, we studied the classic statistical methods like Monte Carlo, FORM, SORM and Kriging. These work well for simple problems, but real design work often involves dozens of uncertain parameters, and the cost of analysis goes up rapidly with the number of variables. This made me curious about how it can be made practical for design decisions. I found a paper by Li and Wang that used an autoencoder, a feedforward network and a Gaussian Process surrogate to reduce the dimensionality of a limit state function. This makes it possible to run reliability calculations in a much smaller space. I decided to implement the full workflow myself in MATLAB to understand how the method works and how accurate it can be.

WHAT I DID

- Recreated the complete dimensional reduction and surrogate modeling method described in the paper
- Built separate modules for data generation, autoencoder training, DFN training, Gaussian Process modeling, and model verification
- Ran 16 case studies on the Griewank function for 10 to 200 dimensions
- Evaluated accuracy, computational cost and convergence across the sample set size of 10 to 4000

HOW IT WORKS

- The autoencoder is trained to represent the data to and from the reduced dimensional space, aka latent space
- A feedforward network is trained to predict this latent representation directly from the input variables
- A Gaussian Process surrogate is built in the latent space to approximate the outputs
- Finally, the feedforward network and surrogate model are used together, to estimate the outputs for a given testing inputs
- The estimated outputs are then compared against the calculated outputs to verify the accuracy

RESULTS

- The predicted data was within 8% of the calculated results for a 10 dimensional problem
- Achieved the computation of a 200 dimensional problem within 0.03 seconds